Лабораторные работы №6–8

Разработка функционала, REST API и деплой программной системы

Дисциплина: «Технологии разработки программного обеспечения»
Форма обучения: заочная

Как устроен этот блок

В ЛР №1–5 был спроектирован учебный проект «Система учёта заявок»: определены требования, архитектура, интерфейс и база данных. В ЛР №6–8 студент превращает проект в работающую систему. ЛР №6 посвящена бизнес-функционалу, ЛР №7 — REST API, ЛР №8 — упаковке и развёртыванию. После ЛР №8 система должна быть готова к следующему блоку, где все последующие лабораторные посвящаются только тестированию.

Важно. Не начинайте новый проект. Все работы №6–8 выполняются в том же Git-репозитории и с той же базой данных, которые использовались в предыдущих лабораторных.

Что должно быть готово до ЛР №6

- Git-репозиторий проекта с историей изменений.
- Техническое задание и список функциональных требований.
- Архитектура приложения и структура каталогов.
- Прототип интерфейса.
- ER-диаграмма и созданная база данных.
- Описание основных сущностей: пользователь, заявка, категория, статус, комментарий/история.

Обязательная работа с нейросетями

В этом блоке нейросеть используется как инструмент разработчика: для анализа требований, проектирования, генерации черновиков, поиска ошибок и ревью. Результат нейросети нельзя принимать без проверки. Все выводы, код и решения остаются ответственностью студента.

Для каждой лабораторной студент выполняет минимум 2 содержательных AI-взаимодействия: одно на анализ/проектирование и одно на генерацию или ревью результата. Можно использовать разные модели и сравнивать ответы.

Единый шаблон AI-журнала

Один шаблон используется во всех лабораторных. Не копируйте его целиком в каждую работу: в отчёте каждой ЛР добавляйте только заполненные строки по своим запросам. Полный шаблон можно вынести в docs/ai_log.md и обновлять в ходе курса.

Поле	Что указать
1. Дата/этап	Когда и на каком этапе разработки использовалась нейросеть.
2. Задача	Что именно нужно было получить или проверить.
3. Модель	Точное название модели и, если доступно, версия/режим.
4. Промпт	Полный текст запроса без сокращений.
5. Контекст	Какие требования, фрагменты кода, схемы или файлы были переданы модели.
6. Результат	Кратко: что предложила модель; при необходимости — скриншот/экспорт диалога.
7. Проверка	Как студент проверил результат: запуском кода, тестом, документацией, сравнением с требованиями.
8. Доработка	Что студент изменил, исправил или полностью отклонил.
9. Почему такая модель/промпт	Почему выбран данный инструмент и почему запрос сформулирован именно так.
10. Итог	Что из ответа реально вошло в проект и чему студент научился.

Правило промпта. Нужно показывать не только удачный финальный запрос, но и хотя бы один неудачный/неполный запрос или первую версию запроса и объяснить, как его улучшили.

Безопасность. Не передавайте нейросети пароли, токены, API-ключи, персональные данные и другие секреты. Используйте тестовые значения.

Лабораторная работа №6. Разработка функционала программной системы

Цель: реализовать основные пользовательские сценарии системы «Система учёта заявок», опираясь на требования, архитектуру и БД, разработанные в ЛР №1–5.

Результат работы

К концу ЛР №6 приложение должно позволять реально выполнять основные операции над заявками, а не только демонстрировать интерфейс или отдельные функции.

Минимальный функционал

Функция	Что должно работать
Создание заявки	Ввод обязательных полей, валидация, сохранение в БД.
Просмотр списка	Получение и отображение списка заявок.
Просмотр одной заявки	Открытие карточки заявки по идентификатору.
Редактирование	Изменение описания, категории, приоритета и других полей по ТЗ.

Функция	Что должно работать
Статус	Изменение статуса заявки с сохранением в БД.
Исполнитель	Назначение исполнителя или ответственного.
Поиск	Поиск по номеру/названию/описанию.
Фильтрация	Фильтрация минимум по статусу и одной дополнительной характеристике.
Удаление/архивация	Реализация в соответствии с ТЗ; опасные действия должны подтверждаться.
Обработка ошибок	Понятные сообщения при неверных данных, отсутствующей записи и ошибке БД.

Порядок выполнения

1. Создайте ветку feature/lab6 в Git и зафиксируйте исходное состояние проекта.

2. Сверьте функции с требованиями из ЛР №1. Отметьте, какие требования будут реализованы в этой работе.

3. Разбейте функционал на небольшие задачи. Например: создание заявки → валидация → сохранение → отображение результата.

4. Реализуйте модели/сущности и доступ к БД в соответствии с архитектурой из ЛР №3.

5. Реализуйте сервисный слой с бизнес-правилами. Не переносите бизнес-логику в SQL и интерфейс.

6. Реализуйте пользовательские сценарии создания, просмотра, редактирования, поиска и изменения статуса заявки.

7. Добавьте обработку ошибок и проверку пользовательского ввода.

8. Заполните БД тестовыми данными: не менее 10 заявок, 4 пользователей, 4 категории и несколько статусов.

9. Проведите ручную проверку всех реализованных сценариев по чек-листу.

10. Сделайте не менее 3 осмысленных commit. В сообщениях commit указывайте, что именно было сделано.

11. Смержите feature/lab6 в develop и обновите README: как запустить приложение локально.

12. Запустите приложение с чистого состояния и убедитесь, что другой человек по README сможет его повторить.

AI-задание

Используйте нейросеть для работы с одной конкретной функциональной задачей. Например: преобразовать требование в пошаговый план реализации, предложить структуру сервиса, объяснить ошибку или провести ревью готовой функции.

1. Сделайте первый промпт широким или недостаточно конкретным. Сохраните его.

2. Проанализируйте ответ и определите минимум 3 недостатка: пропущенное условие, неверное предположение, слишком общий ответ и т.п.

3. Сформулируйте улучшенный промпт, добавив контекст проекта, ограничения и формат ожидаемого ответа.

4. Сравните два ответа и выберите, что реально использовали в проекте.

5. Проверьте предложенный код самостоятельно. Запуск и результат проверки зафиксируйте в AI-журнале по общему шаблону.

Что отправить преподавателю

- Ссылку на Git-репозиторий и ветку develop.
- Исходный код проекта.
- Обновлённый README с инструкцией локального запуска.
- Экспорт/файл тестовых данных или SQL-скрипт наполнения БД.
- Отчёт с описанием реализованных функций и скриншотами.
- Заполненный AI-журнал по общему шаблону для ЛР №6.

Лабораторная работа №7. Разработка REST API

Цель: предоставить функциональность разработанного приложения через HTTP API и научиться документировать, проверять и использовать REST-интерфейсы.

Рекомендуемый вариант

Для Python рекомендуется FastAPI. Студенты, использующие другой язык, реализуют эквивалентный REST API средствами выбранного стека.

Минимальный набор эндпоинтов

Метод	Путь	Назначение	Ожидаемый результат
GET	/requests	Список заявок	HTTP 200 + JSON-массив
GET	/requests/{id}	Получение заявки	HTTP 200 или 404
POST	/requests	Создание заявки	HTTP 201 + созданный объект
PUT/PATCH	/requests/{id}	Изменение заявки	HTTP 200 или 404/422
DELETE	/requests/{id}	Удаление/архивация	HTTP 200/204 или 404
GET	/users	Список пользователей	HTTP 200 + JSON-массив
GET	/statuses	Справочник статусов	HTTP 200 + JSON-массив
GET	/categories	Справочник категорий	HTTP 200 + JSON-массив

Порядок выполнения

1. Создайте ветку feature/lab7.

2. Определите JSON-схемы запросов и ответов для каждой операции.

3. Определите правила валидации: обязательные поля, допустимые значения, длина строк, диапазоны и идентификаторы.

4. Реализуйте API-слой поверх функционала ЛР №6. Не дублируйте бизнес-логику в каждом endpoint.

5. Реализуйте корректные HTTP-коды: 200, 201, 204, 400, 404, 422 и другие, если они действительно нужны.

6. Добавьте обработку ошибок и единообразный формат ошибок.

7. Реализуйте поиск и фильтрацию через query-параметры.

8. Проверьте API минимум 10 запросами в Postman/Insomnia/curl.

9. Сохраните коллекцию запросов и примеры ответов.

10. Проверьте автоматически сгенерированную документацию OpenAPI/Swagger и дополните описания методов.

11. В README добавьте раздел API: запуск, адрес, список основных endpoints и пример запроса.

12. Сделайте минимум 3 осмысленных commit и объедините изменения в develop.

AI-задание

В этой лабораторной задача — научиться использовать нейросеть не только для генерации кода, но и для проектирования интерфейса между компонентами.

1. Попросите модель предложить API-контракт для одной сущности на основе вашего ТЗ.

2. Отдельным промптом попросите модель критически проверить этот контракт: валидацию, HTTP-коды, ошибки, безопасность и удобство клиента.

3. Сравните предложения с тем, что получилось у вас. Минимум одно замечание модели реализуйте или аргументированно отклоните.

4. Приведите в AI-журнале полный промпт, модель, полученный результат, вашу проверку и обоснование окончательного решения.

Что отправить преподавателю

- Git-репозиторий с кодом API.
- Коллекцию Postman/Insomnia или файл с curl-командами.
- Документацию OpenAPI/Swagger или скриншоты доступной документации.
- Таблицу endpoints с методами, параметрами и HTTP-кодами.
- Скриншоты успешных и ошибочных запросов.
- Отчёт и AI-журнал по общему шаблону для ЛР №7.

Лабораторная работа №8. Контейнеризация и деплой приложения

Цель: подготовить приложение к запуску вне локального компьютера, упаковать его в контейнер и выполнить развёртывание на удалённой среде.

Итог ЛР №8

Студент должен получить работающее приложение по удалённому адресу или на учебном сервере и суметь повторить развёртывание по инструкции из README.

1. Яндекс облако
2. Своя машина
3. TimeWeb

Порядок выполнения

1. Создайте ветку feature/lab8.

2. Проверьте, что приложение запускается локально из чистого окружения по инструкции README.

3. Вынесите изменяемые настройки в переменные окружения: режим запуска, порт, строку подключения к БД и другие параметры.

4. Проверьте, что секреты не попадают в Git. Добавьте .env в .gitignore и подготовьте .env.example без реальных секретов.

5. Создайте Dockerfile для приложения.

6. Создайте docker-compose.yml, если для запуска нужны несколько сервисов.

7. Настройте рабочую директорию, установку зависимостей, команду запуска и внешний порт контейнера.

8. Добавьте health-check или endpoint /health, который сообщает, что приложение работает.

9. Соберите Docker-образ локально.

10. Запустите контейнер локально и проверьте основные функции и API.

11. Выберите удалённую среду: учебный сервер, VPS или облачную платформу, поддерживающую Docker.

12. Настройте удалённый запуск и передайте проект на сервер.

13. Выполните миграцию/инициализацию БД и загрузите тестовые данные в соответствии с инструкцией проекта.

14. Проверьте приложение удалённо: открыть интерфейс, выполнить несколько API-запросов, проверить /health.

15. Зафиксируйте адрес развёрнутого приложения, дату деплоя и используемую среду.

16. Добавьте в README полную инструкцию повторного деплоя с нуля.

17. Сделайте минимум 3 осмысленных commit и объедините изменения в develop.

Минимальные артефакты проекта

Файл/артефакт	Назначение
Dockerfile	Описание сборки контейнера приложения
.dockerignore	Исключение ненужных файлов из контекста сборки
docker-compose.yml	Повторяемый локальный запуск связанных сервисов, если нужен
.env.example	Список настроек и переменных окружения без секретов
/health	Простая проверка доступности приложения
README.md	Полная инструкция локального запуска и деплоя

AI-задание

В этой лабораторной студент должен научиться использовать ИИ как технического ревьюера инфраструктуры.

1. Попросите модель предложить Dockerfile для вашего конкретного проекта, указав язык, версию среды, команду запуска и структуру проекта.

2. Затем передайте модели свой Dockerfile и попросите провести ревью: безопасность, размер образа, кеширование зависимостей, порты, права, секреты, health-check.

3. Полученные замечания проверьте самостоятельно. Не принимайте совет только потому, что его дала нейросеть.

4. Как минимум одно замечание исправьте в проекте и покажите результат до/после.

5. В AI-журнале отдельно объясните, почему выбрали модель для этой задачи и какие ограничения задали в промпте.

Что отправить преподавателю

- Git-репозиторий с Dockerfile и файлами деплоя.
- Ссылку на работающую развёрнутую систему или данные для доступа к учебному серверу.
- Скриншоты локального запуска контейнера и удалённого запуска.
- Скриншот успешного обращения к /health и нескольких API-запросов на удалённый адрес.
- README с инструкцией «развернуть проект с нуля».
- Отчёт и AI-журнал по общему шаблону для ЛР №8.

После ЛР №8: переход к тестированию

После завершения этой работы функциональная часть проекта, REST API и деплой считаются подготовленными. Следующий блок лабораторных не должен добавлять новые крупные функции: его задача — системно проверить уже готовый продукт.

- ЛР №9: функциональное и интеграционное тестирование системы, составление тест-кейсов и проверка API.
- ЛР №10: автоматизация тестирования, регрессионный набор, отчёт об ошибках, анализ покрытия и итоговый отчёт о качестве.

Итоговый чек-лист перед сдачей ЛР №8

☐ Основные функции из ЛР №6 работают.

☐ REST API из ЛР №7 работает.

☐ API документировано.

☐ Проект запускается в Docker.

☐ Конфигурация вынесена в переменные окружения.

☐ Секреты отсутствуют в Git.

☐ Есть /health.

☐ Приложение работает в удалённой среде.

☐ README позволяет повторить запуск/деплой.

☐ AI-журнал заполнен по общему шаблону.

☐ Git-история содержит понятные commit.